

Exercise 2

Thinking about types

You've probably noticed by now that GHC will complain if you write some code where the types don't match. To do this, it uses *type inference* to determine the valid types of any functions you write, and checks that everything works. For example, suppose I have a function

```
allTrue :: [Bool] -> Bool
allTrue [] = True
allTrue (x:xs) = x && allTrue xs
```

which checks if every entry of a list of `Bools` is `True`.

This function already exists and is actually called `and` in the standard Prelude.

Now suppose I try and apply this function to a list of integers

```
Prelude> allTrue [1, 2, 3]

<interactive>:7:10: error:
    • No instance for (Num Bool) arising from the literal `1`
    • In the expression: 1
      In the first argument of `allTrue`, namely `[1, 2, 3]`
      In the expression: allTrue [1, 2, 3]
```

Here Haskell has determined that the types are wrong. It is telling me that the deduced type of the argument, which is `Num a => [a]` (that is, a list of numbers), does not satisfy the requirement that `a` is of type `Bool`.

Exercise 2.1

Try this out for yourself.

Reasoning about the types of functions is a core part of writing Haskell. Although GHC can usually infer the type of functions, there are some circumstances where it can need a bit of help. Moreover, type annotations provide useful documentation to the reader of the code. Rather than having to read the body of a function to determine that it takes a list of pairs of `Bool` and `Int` (say), we can just see it in the type definition.

As such, you should *always* annotate your function definitions with their types. GHC will of course check it for you, and complain if your annotation is not valid.

Here are some definitions without types

```
type-snippets.hs
```

```

module Snippets where

x1 = ['a', 'b', 'c']
x2 = ('a', 'b', 'c')
x3 = [(False, 0), (True, 10)]
x4 = ([False, True], [0, 1])
x5 = [tail, reverse, init]

swap (x, y) = (y, x)

pair x y = (x, y)

square x = x*x

palindrome xs = xs == reverse xs

twice f x = f (f x)

```

Exercise 2.2

Download the file and annotate the declarations with their types.

Load it in GHC to check if you got everything right.

Class constraints

You probably noticed that the `palindrome` function required a *class constraint*, namely `Eq`. We'll see much more on these class constraints later.

This makes sense, since to check if a list is palindromic, we need to be able to compare its entries for equality. Try applying `palindrome` to a list of functions.

```

Prelude> palindrome [(+), (-), (+)]
• No instance for (Eq (Integer -> Integer -> Integer))
  arising from a use of `palindrome'
  (maybe you haven't applied a function to enough arguments?)
• In the expression: palindrome [(+), (-), (+)]
  In an equation for `it': it = palindrome [(+), (-), (+)]

```

Here, Haskell complains that it can't check for equality of function types.

Question 2.3

Is Haskell right to complain that it can't check that functions are equal? Explain your reasoning?

Hint for Question 2.3: Think about how you would check if two arbitrary functions (whose source you do not have) might be equal. Think about computable functions and the halting problem.

Some list manipulation

Haskell has particularly strong builtin support for list manipulation (although there are many others too!). We'll look at some simple exercises manipulating lists in various ways.

You can find a template file for these exercises at `lists-exercise2.hs`.

Exercise 2.4

`last :: [a] -> a` returns the final element of a non-empty list. Write a function `butlast :: [a] -> a` which returns the penultimate element of a list of at least two entries.

Try it in two different ways

1. Using existing builtin functions
2. Recursively, using pattern matching.

Question 2.5

Haskell implements lists as linked lists (not arrays). This means that accessing the `nth` element of a list has complexity $O(n)$. Given this information, what is the complexity (in runtime) of your implementation?

If you want to see this in action, you can ask GHC to tell you how long each computation takes by run the command `:set +s`. Here's an example

```
Prelude> :set +s
Prelude> xs = [1..100000000] -- Make a very long list
(0.00 secs, 67,960 bytes)
Prelude> length xs -- Compute its length
100000000
(1.77 secs, 7,200,075,832 bytes)
Prelude> xs !! 1 -- Access the second entry
2
(0.00 secs, 70,120 bytes)
Prelude> xs !! 10000000 -- Access an entry a tenth of the way
through
10000001
(0.19 secs, 720,075,200 bytes)
```

To turn off this extra output, do `:unset +s`.

Exercise 2.6

`tail :: [a] -> [a]` returns the tail of a list, and raise an exception when applied to the *empty list*, `[]`

```
Prelude> tail []  
*** Exception: Prelude.tail: empty list
```

Define a function `safetail :: [a] -> [a]` which maps the empty list to itself and otherwise behaves like `tail`, in three ways:

1. Using a conditional expression;
2. Using guard expressions;
3. Using pattern matching.

In fact, Haskell has a better way of dealing with functions that are not [total](#), using the `Maybe` datatype. We will see this in action later in the course.